

Assignment-5

1. Write a simple base class, then a derived class and use objects of both of them in the main function. It will be a simple illustration of inheritance.

Code:-

```
ques1.cpp
1 //Assignment- 5
2 //Question -1
3 #include <iostream>
4 using namespace std;
5 class Base {
6 public:
7     void show() {
8         cout << "This is Base class" << endl;
9     }
10 };
11 class Derived : public Base {
12 public:
13     void display() {
14         cout << "This is Derived class" << endl;
15     }
16 };
17 int main() {
18     Base b;
19     Derived d;
20
21     b.show();
22     d.show();    // inherited
23     d.display();
24
25     return 0;
26 }
```

Output:-

```
C:\Users\asus\Desktop\oops\ x + v
This is Base class
This is Base class
This is Derived class

-----
Process exited after 0.272 seconds with return value 0
Press any key to continue . . .
```

2. Practice protected access specifier in inheritance. In the base class declare a variable which is protected and access it in the derived class.

Code:-

```

ques1.cpp  [*] ques2.cpp
1  //Assignment- 5
2  //Question -2
3  #include <iostream>
4  using namespace std;
5  class Base {
6  protected:
7      int x;
8  public:
9      void setX(int a) {
10         x = a;
11     }
12 };
13 class Derived : public Base {
14 public:
15     void showX() {
16         cout << "Value of x: " << x << endl;
17     }
18 };
19 int main() {
20     Derived d;
21     d.setX(10);
22     d.showX();
23 }

```

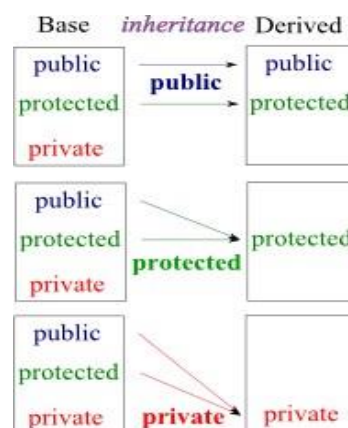
Output:-

```

C:\Users\asus\Desktop\oops\  x  +  v
Value of x: 10
-----
Process exited after 0.2374 seconds with return value 0
Press any key to continue . . .

```

3. Most of the time we use public mode of inheritance, for example *class Derived: public Base{};* Try protected and private access modifiers to understand the difference of various modes of inheritance.



Code:-

```

ques3.cpp  ques2.cpp
1  //Assignment- 5
2  //Question -3
3  #include <iostream>
4  using namespace std;
5
6  class Base {
7  public:
8      int x = 5;
9  };
10 class PublicDerived : public Base {}; // x stays public
11 class ProtectedDerived : protected Base {}; // x becomes protected
12 class PrivateDerived : private Base {}; // x becomes private
13
14 int main() {
15     PublicDerived p;
16     cout << p.x << endl; // accessible
17
18     // ProtectedDerived pr;
19     // cout << pr.x; ? error
20
21     // PrivateDerived pr2;
22     // cout << pr2.x; ? error
23 }

```

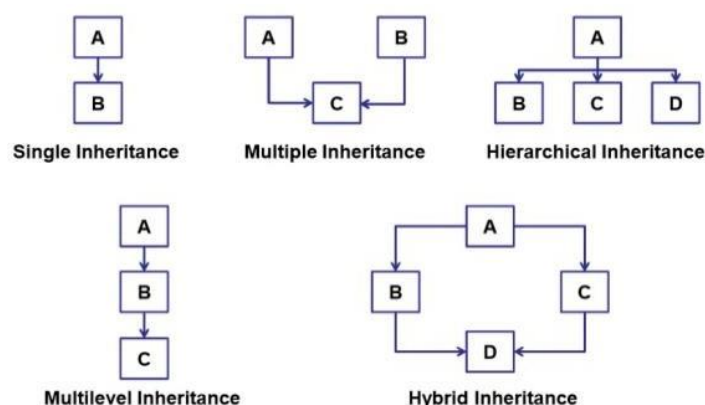
Output:-

```

C:\Users\asus\Desktop\oops\  X  +  v
5
-----
Process exited after 0.2988 seconds with return value 0
Press any key to continue . . .

```

4. Various types of inheritances are as shown below. Write small C++ codes for each inheritance type.



Code:-

```

ques3.cpp  ques2.cpp  ques4.cpp  ques5.cpp
1  //Assignment- 5
2  //Question -4
3  #include <iostream>
4  using namespace std;
5  //Base Class
6  class A {
7  public:
8      void showA() {
9          cout << "Class A (Base Class)" << endl;
10     }
11 };
12 //Single Inheritance
13 class B : public A {
14 public:
15     void showB() {
16         cout << "Class B (Single Inheritance from A)" << endl;
17     }
18 };
19 //Multilevel Inheritance
20 class C : public B {
21 public:
22     void showC() {
23         cout << "Class C (Multilevel Inheritance from B -> A)" << endl;
24     }
25 };
26 //Hierarchical Inheritance
27 class D : public A {
28 public:
29     void showD() {
30         cout << "Class D (Hierarchical Inheritance from A)" << endl;
31     }
32 };
33 //Multiple Inheritance
34 class E {
35 public:
36     void showE() {
37         cout << "Class E (Another Base Class)" << endl;
38     }
39 };
40 class F : public C, public E {
41 public:
42     void showF() {
43         cout << "Class F (Multiple Inheritance from C and E)" << endl;
44     }
45 };
46 int main() {
47     cout << "\n--- Single Inheritance ---\n";
48     B b;
49     b.showA();
50     b.showB();
51
52     cout << "\n--- Multilevel Inheritance ---\n";
53     C c;
54     c.showA();
55     c.showB();
56     c.showC();
57
58     cout << "\n--- Hierarchical Inheritance ---\n";
59     D d;
60     d.showA();
61     d.showD();
62
63     cout << "\n--- Multiple Inheritance ---\n";
64     F f;
65     f.showA();
66     f.showB();
67     f.showC();
68     f.showE();

```

5. How can you use constructors and destructors in C++ inheritance? Write a program to illustrate.

```

ques3.cpp  ques2.cpp  ques4.cpp  ques5.cpp
1  //Assignment- 5
2  //Question -5
3  #include <iostream>
4  using namespace std;
5  class Base {
6  public:
7      Base() {
8          cout << "Base Constructor" << endl;
9      }
10     ~Base() {
11         cout << "Base Destructor" << endl;
12     }
13 };
14 class Derived : public Base {
15 public:
16     Derived() {
17         cout << "Derived Constructor" << endl;
18     }
19     ~Derived() {
20         cout << "Derived Destructor" << endl;
21     }
22 };
23 int main() {
24     Derived d;
25 }
26

```

Output:-

```

C:\Users\asus\Desktop\oops\  x  +  v
Base Constructor
Derived Constructor
Derived Destructor
Base Destructor

-----
Process exited after 0.3304 seconds with return value 0
Press any key to continue . . .

```

6. Implement a base class Book with attributes title, author, and price. Then, create a derived class Textbook that inherits from Book and adds a new attribute subject. Demonstrate how single inheritance is used to manage the data for general books and textbooks.

Code:-

```

ques6.cpp
1 //Assignment- 5
2 //Question -6
3 #include <iostream>
4 using namespace std;
5 class Book {
6 protected:
7     string title, author;
8     float price;
9 public:
10 void input() {
11     cout << "Enter title, author, price: ";
12     cin >> title >> author >> price;
13 }
14 void display() {
15     cout << "Title: " << title << "\nAuthor: " << author << "\nPrice: " << price << endl;
16 }
17 };
18 class Textbook : public Book {
19     string subject;
20 public:
21 void inputTextbook() {
22     input();
23     cout << "Enter subject: ";
24     cin >> subject;
25 }
26 void displayTextbook() {
27     display();
28     cout << "Subject: " << subject << endl;
29 }
30 };
31 int main() {
32     Textbook t;
33     t.inputTextbook();
34     t.displayTextbook();
35 }

```

Output:-

```

C:\Users\asus\Desktop\oops\ x + v
Enter title, author, price: Seas Mathew 200
Enter subject: English
Title: Seas
Author: Mathew
Price: 200
Subject: English

-----
Process exited after 30.22 seconds with return value 0
Press any key to continue . . .

```

7. A software company is creating a program to simulate a car's dashboard. The dashboard needs to display speed, fuel level, and temperature. The speed is controlled by a Speedometer class, the fuel level by a FuelGauge class, and the temperature by a Thermometer class. Implement the three classes: Speedometer, FuelGauge, and Thermometer, each with relevant attributes and methods. Then, create a CarDashboard class that inherits from all three classes to display the combined information on the dashboard. Demonstrate how multiple inheritance is used to build this class.

Code:-

```

ques6.cpp  [*] Untitled1
1  //Assignment- 5
2  //Question -7
3  #include <iostream>
4  using namespace std;
5  class Speedometer {
6  protected:
7      int speed;
8  public:
9      void setSpeed(int s) { speed = s; }
10 };
11 class FuelGauge {
12 protected:
13     int fuel;
14 public:
15     void setFuel(int f) { fuel = f; }
16 };
17 class Thermometer {
18 protected:
19     int temp;
20 public:
21     void setTemp(int t) { temp = t; }
22 };
23 class CarDashboard : public Speedometer, public FuelGauge, public Thermometer {
24 public:
25     void display() {
26         cout << "Speed: " << speed << endl;
27         cout << "Fuel: " << fuel << endl;
28         cout << "Temperature: " << temp << endl;
29     }
30 };
31 int main() {
32     CarDashboard c;
33     c.setSpeed(80);
34     c.setFuel(50);
35     c.setTemp(90);
35     c.setTemp(90);
36     c.display();
37 }

```

Output:-

```

C:\Users\asus\Desktop\oops\  X  +  v
Speed: 80
Fuel: 50
Temperature: 90

-----
Process exited after 0.2702 seconds with return value 0
Press any key to continue . . . |

```

8. You are tasked with creating a system for a library that tracks different types of users. The system needs to handle general user information such as name, ID, and contact details. There are two specific types of users: Student and Teacher. Each type of user has additional attributes, such as grade level for students and department for teachers. Implement a base class LibraryUser with general attributes. Then, create two derived classes Student and Teacher that inherit from LibraryUser and add their own specific attributes. Demonstrate how hierarchical inheritance is applied in this scenario.

Code:-

```
ques6.cpp  ques7.cpp  ques8.cpp
1  //Assignment- 5
2  //Question -8
3  #include <iostream>
4  using namespace std;
5  class LibraryUser {
6  protected:
7      string name;
8      int id;
9      string contact;
10 public:
11     void inputUser() {
12         cout << "Enter Name: ";
13         cin >> name;
14         cout << "Enter ID: ";
15         cin >> id;
16         cout << "Enter Contact: ";
17         cin >> contact;
18     }
19     void displayUser() {
20         cout << "Name: " << name << endl;
21         cout << "ID: " << id << endl;
22         cout << "Contact: " << contact << endl;
23     }
24 };
25 class Student : public LibraryUser {
26     int grade;
27 public:
28     void inputStudent() {
29         inputUser();
30         cout << "Enter Grade: ";
31         cin >> grade;
32     }
33     void displayStudent() {
34         displayUser();
35         cout << "Grade: " << grade << endl;
```

```

33 void displayStudent() {
34     displayUser();
35     cout << "Grade: " << grade << endl;
36 }
37 };
38 class Teacher : public LibraryUser {
39     string department;
40 public:
41     void inputTeacher() {
42         inputUser();
43         cout << "Enter Department: ";
44         cin >> department;
45     }
46     void displayTeacher() {
47         displayUser();
48         cout << "Department: " << department << endl;
49     }
50 };
51 int main() {
52     Student s;
53     Teacher t;
54     cout << "\n--- Enter Student Details ---\n";
55     s.inputStudent();
56     cout << "\n--- Enter Teacher Details ---\n";
57     t.inputTeacher();
58     cout << "\n--- Student Details ---\n";
59     s.displayStudent();
60     cout << "\n--- Teacher Details ---\n";
61     t.displayTeacher();
62 }

```

Output:-

```

C:\Users\asus\Desktop\oops\  ×  +  v

--- Enter Student Details ---
Enter Name: Raju
Enter ID: 101
Enter Contact: 9898989898
Enter Grade: A

--- Enter Teacher Details ---
Enter Name: Enter ID: Enter Contact: Enter Department:
--- Student Details ---
Name: Raju
ID: 101
Contact: 9898989898
Grade: 0

--- Teacher Details ---
Name:
ID: 4254569
Contact:
Department:

-----
Process exited after 15.72 seconds with return value 0
Press any key to continue . . .

```

9. A logistics company needs a software system to manage its vehicle fleet. All vehicles share common attributes like make, model, and year. Trucks have additional attributes like load_capacity. Furthermore, refrigerated trucks have a special attribute called temperature_control. Implement a base class Vehicle with common attributes. Then, create a derived class Truck that adds the load_capacity attribute. Finally, create another derived class RefrigeratedTruck that inherits from Truck and adds the temperature_control attribute. Demonstrate how multilevel inheritance works in this case.

Code:-

```
ques6.cpp ques7.cpp ques8.cpp ques9.cpp
1  #include <iostream>
2  using namespace std;
3  class Vehicle {
4  protected:
5      string make, model;
6      int year;
7  public:
8      void inputVehicle() {
9          cout << "Enter Make: ";
10         cin >> make;
11         cout << "Enter Model: ";
12         cin >> model;
13         cout << "Enter Year: ";
14         cin >> year;
15     }
16     void displayVehicle() {
17         cout << "Make: " << make << endl;
18         cout << "Model: " << model << endl;
19         cout << "Year: " << year << endl;
20     }
21 };
22 class Truck : public Vehicle {
23 protected:
24     int load_capacity;
25 public:
26     void inputTruck() {
27         inputVehicle();
28         cout << "Enter Load Capacity: ";
29         cin >> load_capacity;
30     }
31     void displayTruck() {
32         displayVehicle();
33         cout << "Load Capacity: " << load_capacity << " kg" << endl;
34     }
35 };
36
37 class RefrigeratedTruck : public Truck {
38     int temperature_control;
39 public:
40     void inputRefrigeratedTruck() {
41         inputTruck();
42         cout << "Enter Temperature Control (in °C): ";
43         cin >> temperature_control;
44     }
45     void displayRefrigeratedTruck() {
46         displayTruck();
47         cout << "Temperature Control: " << temperature_control << " °C" << endl;
48     }
49 };
50
51 int main() {
52     RefrigeratedTruck rt;
53     cout << "\nEnter Refrigerated Truck Details\n";
54     rt.inputRefrigeratedTruck();
55     cout << "\nRefrigerated Truck Details\n";
56     rt.displayRefrigeratedTruck();
57 }
```

Output:-

```

C:\Users\asus\Desktop\oops\  X  +  v

Enter Refrigerated Truck Details
Enter Make: Mahindra
Enter Model: M202
Enter Year: 2024
Enter Load Capacity: 12
Enter Temperature Control (in °C): 100

Refrigerated Truck Details
Make: Mahindra
Model: M202
Year: 2024
Load Capacity: 12 kg
Temperature Control: 100 °C

-----
Process exited after 22.67 seconds with return value 0
Press any key to continue . . .

```

10. You are developing a software system for an academic institution. The institution has various roles like Person, Staff, and Student. A Person has general attributes like name and address. Staff members, who are a type of Person, have additional attributes like employee_id and department. Student, another type of Person, has attributes like student_id and grade. Some Staff members are also Students (e.g., teaching assistants) and need to inherit from both classes. Implement a base class Person with general attributes. Then, create derived classes Staff and Student that inherit from Person and add their specific attributes. Finally, create a TeachingAssistant class that inherits from both Staff and Student. Demonstrate how hybrid inheritance is applied and managed in this scenario.

Code:-

```

ques6.cpp  ques7.cpp  ques8.cpp  [*] ques9.cpp  ques10.cpp

1 //Assignment- 5
2 //Question -10
3 #include <iostream>
4 using namespace std;
5 class Person {
6 protected:
7     string name, address;
8 public:
9     void inputPerson() {
10         cout << "Enter Name: ";
11         cin >> name;
12         cout << "Enter Address: ";
13         cin >> address;
14     }
15     void displayPerson() {
16         cout << "Name: " << name << endl;
17         cout << "Address: " << address << endl;
18     }
19 };
20 class Staff : virtual public Person {
21 protected:
22     int employee_id;
23     string department;
24 public:
25     void inputStaff() {
26         cout << "Enter Employee ID: ";
27         cin >> employee_id;
28         cout << "Enter Department: ";
29         cin >> department;
30     }
31     void displayStaff() {
32         cout << "Employee ID: " << employee_id << endl;
33         cout << "Department: " << department << endl;
34     }
35 };

```

```

ques6.cpp  ques7.cpp  ques8.cpp  [*] ques9.cpp  ques10.cpp
34 |     }
35 | };
36 | class Student : virtual public Person {
37 | protected:
38 |     int student_id;
39 |     string grade;
40 | public:
41 |     void inputStudent() {
42 |         cout << "Enter Student ID: ";
43 |         cin >> student_id;
44 |         cout << "Enter Grade: ";
45 |         cin >> grade;
46 |     }
47 |     void displayStudent() {
48 |         cout << "Student ID: " << student_id << endl;
49 |         cout << "Grade: " << grade << endl;
50 |     }
51 | };
52 | class TeachingAssistant : public Staff, public Student {
53 | public:
54 |     void inputTA() {
55 |         inputPerson();
56 |         inputStaff();
57 |         inputStudent();
58 |     }
59 |     void displayTA() {
60 |         cout << "\nTeaching Assistant Details\n";
61 |         displayPerson();
62 |         displayStaff();
63 |         displayStudent();
64 |     }
65 | };
66 | int main() {
67 |     TeachingAssistant ta;
68 |     cout << "\nEnter Teaching Assistant Details\n";

```

```

65 | };
66 | int main() {
67 |     TeachingAssistant ta;
68 |     cout << "\nEnter Teaching Assistant Details\n";
69 |     ta.inputTA();
70 |     ta.displayTA();
71 | }

```

Output:-

```

C:\Users\jasus\Desktop\oops\  X  +  v
Enter Teaching Assistant Details
Enter Name: Mehul
Enter Address: Gwalior
Enter Employee ID: 101
Enter Department: Csdd
Enter Student ID: 123
Enter Grade: A

Teaching Assistant Details
Name: Mehul
Address: Gwalior
Employee ID: 101
Department: Csdd
Student ID: 123
Grade: A

-----
Process exited after 26.35 seconds with return value 0
Press any key to continue . . .

```